

Laboratorio 7

Introducción - El problema del laberinto.

El problema del laberinto mediante backtracking (vuelta atrás) consiste en encontrar un camino desde un punto de inicio hasta una salida en una cuadrícula llena de paredes, probando rutas posibles de manera iterativa y deshaciendo el camino cuando se llega a un callejón sin salida.

posición de inicio (0,0), posición de salida(4,6), valor 0 en el laberinto (posición vacía), valor -1 en el laberinto (muro), números en color rojo (ruta de éxito)

LABERINTO (4,6)

1	2	-1	0	0	0
0	3	4	5	6	7
0	0	-1	0	0	8
0	0	0	0	0	9

Enunciado general

- Tablero: Una cuadrícula de (M x N) casillas.
- Posición inicial: Se elige casilla de inicio con coordenadas (0, 0).
- Movimiento - **Paso a paso**: El programa avanza a una celda vecina (arriba, abajo, izquierda, derecha) si es un espacio libre. **Marcaje temporal**: La celda actual se marca como visitada para no entrar en ciclos infinitos. **Vuelta atrás**: Si desde esa nueva posición no hay salida, el algoritmo borra la marca o indica que no sirve (backtracking) y prueba con otra dirección disponible. **Condición de parada**: Se detiene con éxito al alcanzar la casilla de salida o devuelve falso si se probaron todas las opciones sin éxito.
- Objetivo: Encontrar una secuencia de movimientos permitidos para el laberinto que permita encontrar la salida.

Definición del problema y requisitos de la solución

El objetivo es implementar en Python un algoritmo que permita encontrar solución (si es que la hay) , para un Laberinto cuyo tamaño (filas y columnas), número de obstáculos, y coordenadas de cada obstáculo se encuentran en el archivo (laberinto.dat).

Se pide:



1. Implementar el algoritmo que permita saber si existe o no solución al laberinto (laberinto.dat), y si la hay que guarde la solución en un archivo (solucion.txt). (50 % nota)
2. Implementar el algoritmo que permita encontrar todas las soluciones (si existen) al problema del laberinto (laberinto.dat). (30 % nota)
Todas las soluciones se deben guardar en el archivo (solucion.txt).
3. Implementar el algoritmo que permita encontrar la mejor solución (si existe) al problema del laberinto (laberinto.dat). (20 % nota)
Todas las soluciones se deben guardar en el archivo (solucion.txt).
La mejor solución se debe guardar en otro archivo (mejor_solución.dat)

Plantilla Backtracking iterativo - una solución

```
begin
    #definición particular de cada problema
    while(tenga candidatos AND not solución)do
        if(valido movimiento elegido)then
            avanzar en la búsqueda de la solución
            if(llegó al final) then
                solucion = true
            else
                dejar pistas para devolverme
            endif
        else
            siguiente candidato
            while(no estoy en el inicio AND ya no tengo candidatos)do
                retrocedo
            endwhile
        endif
    endwhile
end
```

Instrucciones

1. El laboratorio deberá ser desarrollado de manera **individual** utilizando Python.
2. La solución deberá implementarse en un archivo con extensión **.py**, el cual deberá contener el algoritmo solicitado de acuerdo con los requisitos definidos en el laboratorio.
3. El programa deberá ejecutarse correctamente, sin presentar errores de compilación o ejecución, y cumplir con todas las funcionalidades especificadas para el algoritmo.

4. El código deberá seguir los principios de la programación estructurada, utilizando adecuadamente variables, funciones, estructuras de control y demás elementos vistos durante el curso.
5. Se deberán considerar los casos de prueba necesarios para verificar el correcto funcionamiento del programa, incluyendo la validación de las entradas cuando corresponda.
6. Utilizar como base de trabajo Pseudo código en el ANEXO

Evaluación.

1. La entrega del laboratorio será **individual** y deberá realizarse dentro del plazo establecido por el docente.
2. La evaluación se regirá por la **Pauta de Evaluación de Laboratorio** correspondiente a la asignatura, considerando tanto el correcto funcionamiento del programa como el cumplimiento de los requisitos establecidos.
3. Como parte de la evaluación, el estudiante deberá realizar una **defensa del código desarrollado**, en la cual deberá explicar la lógica de la solución implementada, el funcionamiento del algoritmo y las decisiones tomadas durante su desarrollo.
4. Durante la defensa, el docente podrá realizar preguntas relacionadas con el código presentado y solicitar modificaciones o explicaciones sobre partes específicas del programa, con el fin de verificar la comprensión del estudiante y la autoría del trabajo entregado.
5. Todos los trabajos serán evaluados en el laboratorio
6. **FECHA DE ENTREGA: 27 DE SEPTIEMBRE DE 2026 HASTA LAS 24 HORAS.**

ANEXO

```
#####
BEGIN PRINCIPAL
    #MxN DEFINE EL TAMAÑO DEL LABERINTO
    LABERINTO[M][N]
    CREAR_TABLERO(LABERINTO,M,N)
    IF(BUSCA_SOLUCIÓN(LABERINTO,M,N))THEN
        IMPRIMIR_LABERINTO(LABERINTO,M,N)
    ELSE
        WRITE("NO HAY SOLUCION")
    ENDIF
END
#####
BEGIN CREAR_TABLERO(LABERINTO,M,N)
    FOR I=0;I<M;I++
        FOR J=0;J<N;J++
            LABERINTO[I][J] = 0
        ENDFOR
    ENDFOR
    LABERINTO[0][2] = -1
    LABERINTO[1][2] = -1
    LABERINTO[2][2] = -1
    LABERINTO[2][4] = -1
    LABERINTO[3][4] = -1
END
#####
BEGIN BUSCA_SOLUCION(LABERINTO,M,N)
    OPCION = 1; SOLUCION=TRUE;
    XINI = 0; YINI=0; X=XINI;Y=YINI;XFIN=M-1;YFIN=N-1;CONTADOR = 1
    LABERINTO[X][Y]=CONTADOR
    CREAR_AUXILIAR(LABERINTO_AUX,M,N)
    WHILE(OPCION<=4 AND NOT SOLUCION)DO
        IF(VÁLIDO(LABERINTO,M,N,X,Y,OPCION))THEN
            CONTADOR = CONTADOR + 1
            NX,NY = BUSCA_NX_NY(LABERINTO,M,N,OPCION)
            LABERINTO[NX][NY]=CONTADOR
            IF (NX == XFIN AND NY == YFIN)THEN
                RETURN TRUE
            ELSE
                LABERINTO_AUX[X][Y]=OPCION
                X = NX; Y = NY; OPCION = 1
            ENDIF
        ENDIF
    ENDWHILE
END
```



```
ELSE
    OPCION = OPCION + 1
    WHILE((X != XINI AND Y != YINI) AND OPCION = 5)DO
        LABERINTO[X][Y]=0
        CONTADOR = CONTADOR - 1
        NX,NY = BUSCA_POSICION(LABERINTO,M,N,CONTADOR)
        OPCION = LABERINTO_AUX[NX][NY] +1
        LABERINTO_AUX[NX][NY] = 0
        X = NX ; Y = NY
    ENDWHILE
ENDIF
ENDWHILE
RETURN FALSE
END
#####
BEGIN BUSCA_NX_NY(LABERINTO,M,N,OPCION)
    IF(OPCION == 1)THEN
        RETURN X,Y+1
    ENDIF
    IF(OPCION == 2)THEN
        RETURN X+1,Y
    ENDIF
    IF(OPCION == 3)THEN
        RETURN X,Y-1
    ENDIF
    IF(OPCION == 4)THEN
        RETURN X-1,Y
    ENDIF
END
END
#####
BEGIN BUSCA_POSICION(LABERINTO,M,N,CONTADOR)
    FOR I=0;I<M;I++
        FOR J=0;J<N;J++
            IF (LABERINTO[I][J] == CONTADOR)THEN
                RETURN TRUE
            ENDIF
        ENDFOR
    ENDFOR
    RETURN FALSE
END
#####
BEGIN VALIDO(LABERINTO, M,N,X,Y,OPCION)
    IF(OPCION == 1 AND LABERINTO[X][Y+1]==0 AND Y+1 <N)THEN
        RETURN TRUE
    
```



```
ENDIF
IF(OPCION == 2 AND LABERINTO[X+1][Y]==0 AND X+1 <M)THEN
    RETURN TRUE
ENDIF
IF(OPCION == 3 AND LABERINTO[X][Y-1]==0 AND Y-1 >=0)THEN
    RETURN TRUE
ENDIF
IF(OPCION == 4 AND LABERINTO[X-1][Y]==0 AND X-1 >=0)THEN
    RETURN TRUE
ENDIF
RETURN FALSE
END
#####
```